

Akka vs. PydanticAI

A comparison for teams building agentic AI

July 2026

PydanticAI gives you a clean, type-safe way to build agents in Python; Akka gives you a platform that runs and guarantees them in production — and Akka Specify can deliver and run the entire governed system for you as a guaranteed outcome. PydanticAI is a typed Python agent framework — model-agnostic, with structured outputs, dependency injection, and graph support. It is a framework, not a platform: no durable runtime, no HA/DR, no embedded governance, no operational SLA, and no delivery model. You assemble and operate production yourself, or contract a labor-led integration engagement to do it for you.

99.9999%

AKKA AVAILABILITY SLA

Weeks

AKKA SPECIFY DELIVERY

Fixed

ONE ALL-IN PRICE

90%

LESS INFRASTRUCTURE

DIMENSION	PYDANTICAI	AKKA
What it is	A typed Python agent framework (library)	A full-stack agentic systems platform
How you reach production	Self-build, or contract a labor-led integration engagement	Build with spec-driven development, or Akka Specify delivers and runs it
Commercial model	Framework is free; billed for a durable-execution engine, Logfire, infrastructure, and build-and-operate labor	One fixed price — platform, infrastructure, tokens, training, delivery, and operations
Outcome guarantee	Effort only; the outcome is not guaranteed	The delivered outcome is guaranteed
Scope	Agent construction, structured output, tools, MCP, graph; durability, HA/DR, governance, and operations are the customer's	Orchestration, agents, memory, streaming, APIs, observability, and governance on one runtime
Durable execution	None native — wrapped around external engines (Temporal, DBOS, Prefect, Restate)	Built into the runtime; event-sourced, replayable
Availability SLA	None published for the framework; contractual SLA only on Logfire Enterprise observability	99.9999% — entire platform, backed by indemnities
RTO / RPO	Owned by the customer's chosen infrastructure	Sub-1-minute RTO; zero-byte RPO
Memory / state	Process memory; durability via the external engine's database	Durable in-memory, 4ms reads / sub-10ms writes
Governance / EU AI Act	Observability and evals via Logfire (after-the-fact); no inline enforcement or classification	Aspect-woven runtime enforcement + full pre-production governance
Real-time streaming	Not provided; sourced separately	Built in, backpressured, petabyte-scale, no external broker
Cost model	Python infrastructure footprint you provision and operate	Shared compute; up to 90% lower infrastructure for the same workload
Improves after go-live	Logfire detects and surfaces quality issues; improvement is manual	Continuous evaluation, reinforcement learning, and distillation — up to 80% lower token cost over time
Certifications	SOC 2 Type II, HIPAA, GDPR (Logfire)	19 standards (SOC 2 II + public SOC 3, ISO 27001/42001, HIPAA, PCI DSS, GDPR, NIS2, DORA, EU

1. PydanticAI Is a Framework; Akka Is a Platform

PydanticAI is a typed Python agent framework from the team behind Pydantic. Its design point is developer experience: model-agnostic agents, structured and validated outputs, dependency injection, MCP support, and graph-based multi-agent workflows, all checked by your type checker and IDE. For teams building agents in Python who want errors caught at write-time, that DX is genuinely clean. A framework defines how you write code; a platform runs it and guarantees it. The durable runtime, high availability, disaster recovery, streaming, the API tier, embedded governance, and the operational SLA are **not in the framework** — you source, integrate, and operate them, and own every failure across the seams.

Capability	PydanticAI	Akka
Typed agent construction (tools, structured output, MCP, graph)	Yes — clean, type-safe	Yes
Durable runtime	Delegated to external engines	Built in
High availability / disaster recovery	Customer-owned	Built in, active-active
Durable memory	Customer-owned (external engine's DB)	Built in, 4ms / sub-10ms
Real-time streaming	Not provided	Built in, backpressured, petabyte-scale
HTTP / gRPC API layer	Not provided	Built in
Governance / policy enforcement	Observe and evaluate (Logfire)	Inline, runtime-embedded
Operational SLA	None on the framework	99.9999%, entire platform

2. Two Ways to Production: Build It, or Have It Delivered

Akka offers two ways onto one platform: build the system yourself with spec-driven development, or have **Akka Specify** deliver and operate the entire governed system for you as a guaranteed outcome — in weeks, for one fixed price. PydanticAI offers neither a platform nor a delivery model: a PydanticAI application is the customer's to build, run, and own.

	With PydanticAI	With Akka Specify
Model	Self-build, or a labor-led integration engagement	You provide the specifications
Who does the work	Your engineers, or forward-deployed / staff-augmentation contractors	Akka generates, governs, delivers, and runs the system
Timeline	Months, by construction — plus selecting and integrating a durable-execution engine	Weeks, not quarters
Billing	Framework is free; billed for a durable-execution engine, Logfire, infrastructure, and build-and-operate labor	One fixed price
Afterward	You inherit the system and its operations	Kept up, safe, and improving — operated by Akka
Guarantee	Effort is guaranteed; the outcome is not	The delivered outcome is guaranteed

PydanticAI publishes no shared-responsibility model, because it publishes no operational commitment to begin with: the framework, whichever durable-execution engine you choose, and the observability you wire in through Logfire are each the customer's to assemble and run, and none of it is delivered as a system. A team that cannot self-build contracts a labor-led integration engagement — billed for time and materials, guaranteeing effort rather than the outcome, and handing back a system the team then owns and operates. Akka Specify inverts that: you provide a handful of plain-language specifications, and Akka generates, tests, governs, deploys, and runs the system — then keeps it available, safe, and improving under one agreement.

3. Reliability, SLA, and Durability

PydanticAI publishes no availability SLA, because a framework does not run anything — availability is a property of the infrastructure the customer builds around it. For durability, PydanticAI leans on external durable-execution engines: it ships co-maintained integrations with **Temporal, DBOS, Prefect, and Restate**, each of which checkpoints or replays state in its own datastore. The framework documents this explicitly; durability is delegated, not native. Reliability, recovery, and the SLA for the running system are owned and operated by the customer and their chosen engines. Akka delivers durability and availability as runtime properties: state is event-sourced and replayable from the journal, recovery is automatic, and the platform carries a contractual six-nines SLA.

Metric	PydanticAI	Akka
Availability SLA	None on the framework; SLA only on Logfire Enterprise	99.9999% — entire platform
Who operates it	The customer (or an inherited integrator)	Akka SREs, 24/7
Allowed downtime / year	Whatever the assembled stack delivers	~31 seconds
RTO	Owned by the customer's chosen engine	Sub-1 minute
RPO	Owned by the customer's chosen engine	Zero byte
Durable execution	Delegated to Temporal / DBOS / Prefect / Restate	Native, in the runtime
SLA scope	None	The entire platform

4. Up to 90% Cheaper to Operate — and It Keeps Getting Cheaper

AI systems built with Akka are up to **90% cheaper to operate** than Python-based systems — a function of the infrastructure required for the same agentic transaction volume, not list price. A PydanticAI deployment carries the Python infrastructure footprint, plus the separate datastore and operations of whichever durable-execution engine provides recovery, plus the memory, streaming, API, observability, and governance tiers run alongside it.

Akka runs all of it on one shared-compute runtime (actor concurrency ~10T tokens/core/year vs ~2T; ~80% less compute than Python frameworks; Manulife: up to 300% more concurrency, 30–50% faster processing after porting from Python), shared compute, and micro-checkpointing. The delivered outcome also **compounds**: Akka Verify's continuous evaluation, reinforcement learning, and distillation cut token cost up to 80% over time — tuning Logfire can detect but not ship. The spend is predictable — **one fixed price**, not a Python infrastructure footprint that scales with load plus separate build-and-operate labor.

6. Governance and the EU AI Act

PydanticAI's governance story is observability. Through Pydantic Logfire — the team's commercial product — you get tracing, evaluations, and agent monitoring, with SOC 2 Type II, HIPAA, and GDPR for the observability service and an EU data region. That is real, and it matters for debugging and quality. It is observe-and-evaluate, applied to logs after execution: it does not enforce policy inline, pause or override a running process, classify a system before it ships, produce an immutable hash-chained interaction ledger, or seal an audit artifact. Those obligations remain the customer's.

The penalties are enforceable now

Violation	Maximum Fine
Prohibited AI practices (Art. 5)	EUR 35M or 7% global turnover
High-risk obligations (Art. 9-15)	EUR 15M or 3% global turnover

High-risk AI carries a 10-year logging-retention obligation (Art. 72).

5. Maturity and Vendor

PydanticAI reached **V1 in September 2025**; the current stable release is 1.107.0 (June 2026), marked Production/Stable, with V2 in beta. The framework is young but real. Pydantic the company is venture-funded — a **\$12.5M Series A led by Sequoia** (Oct 2024), ~\$17.2M total — and monetizes through Logfire.

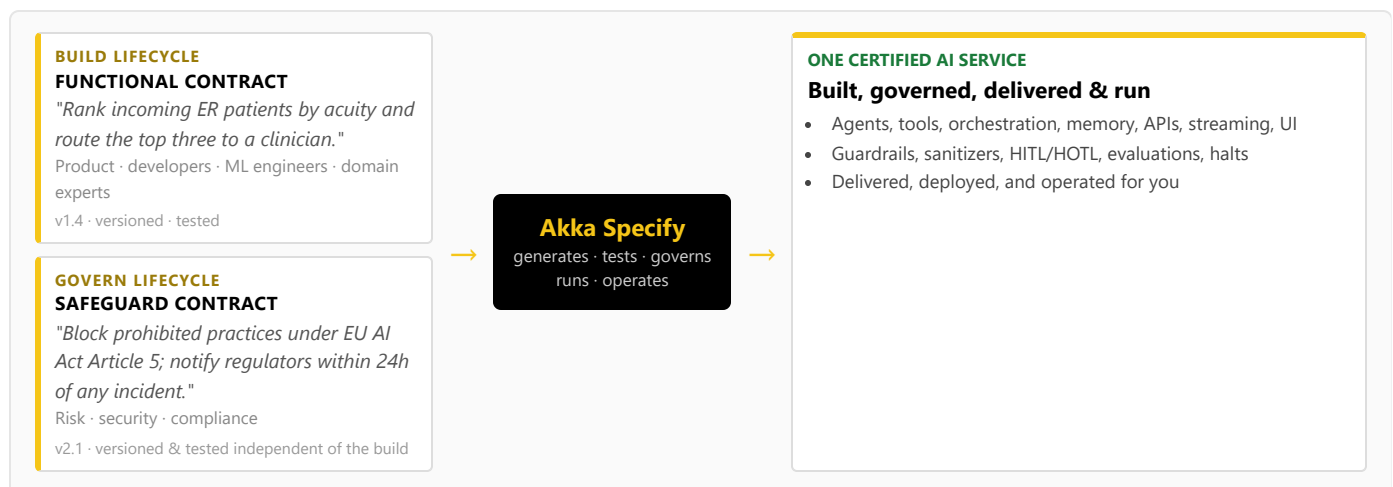
Akka is profitable and self-funding, with 18 years, 100,000+ production deployments, and 52 banks behind it. Dell Technologies Capital is the largest shareholder, a customer, and an AI partner — so the platform you standardize on does not depend on a venture timeline.

How Akka governs

At the runtime: inline guardrails, policies, LLMs-as-a-judge, and sanitizers; hash-chained immutable evidence; HITL/HOTL control; classification against 186 regulations and 877 controls (288 carrying a financial penalty) before a system ships; multi-persona sign-offs; a sealed Governance Posture Package; and Akka Verify proving conformance from the running system. Governance a PydanticAI team would assemble and bolt on, Akka enforces inline — and delivers it already wired when the system is built through Akka Specify.

7. One Certified System — Built, Governed, Delivered, and Run

Building with PydanticAI means engineers writing Python agent code; there is no built-in path for a product manager, domain expert, or risk officer to contribute, and no built-in governance lifecycle. Akka runs two independent lifecycles on one platform via **Akka Specify**:



Akka Verify  validates the running system against both specs and fine-tunes the AI from production data.

Akka generates, tests, governs, **runs, and operates** one certified AI service that satisfies both contracts, and **Akka Verify** validates the running system against both. The build and governance lifecycles are versioned and tested independently, by different audiences, and — through Akka Specify — the whole certified system is delivered and operated as a guaranteed outcome. PydanticAI has no equivalent for either the independent governance lifecycle or the delivery model.

8. Real-Time Streaming at Petabyte Scale

PydanticAI has no streaming engine; real-time pipelines are provisioned separately. Akka's streaming is built into the runtime — continuous, backpressured, **petabyte-scale, in-memory**, with no external broker — powering both agent feedback loops and high-throughput data processing (the engine behind Tubi's real-time hyper-personalization at 5 billion tokens per second).

9. For the Buyer: Risk, Compliance, and Accountability

Buyer concern	PydanticAI	Akka
Certifications & audits	SOC 2 Type II, HIPAA, GDPR (Logfire observability)	19 standards — SOC 2 II + public SOC 3, ISO 27001/42001, HIPAA, PCI DSS, GDPR, NIS2, DORA, EU AI Act, NIST AI RMF — plus annual pen tests, SBOMs, 40+ policies (trust.akka.io)
Delivery & outcome	Self-build or a time-and-materials integrator; effort is guaranteed, the outcome is not	Akka Specify delivers and operates the system for one fixed price; the outcome is guaranteed
Scope of accountability	The framework; you integrate and operate the runtime, durability, and everything else	One platform, one SLA, 24/7 SRE — Akka owns the running system
Risk transfer	Standard open-source / cloud terms	Availability and data-integrity guarantees backed by contractual indemnities
Track record & funding model	Venture-funded: \$12.5M Series A led by Sequoia (Oct 2024), ~\$17.2M total; monetizes via Logfire	Profitable and self-funding; 18 years and 100,000+ deployments (52 banks); Dell Technologies Capital is largest shareholder, a customer, and an AI partner
Budget predictability	Python infrastructure footprint you size and operate	One fixed price finance can forecast

PydanticAI gives you a clean, type-safe way to author agents in Python. Akka gives you the platform that runs them: durable runtime, high availability, embedded governance, and an SLA Akka owns. The decision is scope and accountability — a framework you build a production system around, versus the production system, which, through Akka Specify, Akka will deliver and run for you.

Customers Running Agentic and Real-Time Systems on Akka

Manulife 2,000 developers across 100 projects on one governed platform	Tubi 5B tok/s real-time hyper-personalization engine	Swiggy 71ms order-assignment AI, ~50% latency reduction	John Deere 1,000+ tractor sensors turned into real-time insight	Verizon 750% order-processing capacity gain; 6s → 2.4s response
--	--	---	---	---

Common Questions

We don't have a team to build this — what are our options?

Two. Build it on the platform with spec-driven development, or have Akka Specify deliver and operate it for you. You provide plain-language specifications; Akka generates, governs, delivers, and runs the system as a guaranteed outcome, for one fixed price. With PydanticAI, production is self-build — assembling the framework, a durable-execution engine, and Logfire — or a labor-led integration engagement you then own.

How is Akka Specify different from hiring an integrator to build our PydanticAI app?

An integration engagement sells effort — time-and-materials over months — and hands back a system you own and operate; the outcome

PydanticAI supports durable execution. Doesn't that cover reliability?

PydanticAI integrates with external durable-execution engines — Temporal, DBOS, Prefect, and Restate — and its own docs frame durability as delegated to those systems. You select, provision, and operate a separate engine and its datastore, and own the SLA, HA/DR, and recovery across the seams. Akka's durability is native to the runtime: event-sourced, replayable, sub-1-minute RTO, zero-byte RPO, under a six-nines SLA.

Can we add governance on top of PydanticAI with Logfire?

is not guaranteed. Akka Specify sells the outcome: a governed system delivered in weeks for one fixed price, then kept up, safe, and improving by Akka. You own the specifications, not the operational burden.

We like PydanticAI's typed developer experience. Why add Akka?

PydanticAI's type-safe DX is a real strength for authoring agents in Python. It is a framework, so the durable runtime, high availability, disaster recovery, streaming, the API tier, embedded governance, and the operational SLA are yours to build and run. Akka delivers those as one platform with a 99.9999% SLA, and teams can keep their typed agent design discipline while moving production onto a runtime that guarantees it.

Sources

PydanticAI — what it is: ai.pydantic.dev, pydantic.dev/pydantic-ai — model-agnostic, type-safe Python agent framework; structured outputs, dependency injection, MCP, graph workflows

PydanticAI durable execution: ai.pydantic.dev/durable_execution/overview — co-maintained integrations with Temporal, DBOS, Prefect, Restate; durability delegated to external engines

PydanticAI maturity: github.com/pydantic/pydantic-ai/releases, pypi.org/project/pydantic-ai — V1 since Sept 2025; stable 1.107.0 (June 10, 2026); V2 in beta; "Production/Stable"

Pydantic Logfire: pydantic.dev/logfire, logfire.pydantic.dev/docs/compliance — tracing and evals; SOC 2 Type II, HIPAA, GDPR, EU data region; SLA on Enterprise plan

Pydantic funding: techcrunch.com (Oct 1, 2024), crunchbase.com — \$12.5M Series A led by Sequoia (Oct 2024); ~\$17.2M total over two rounds

Akka trust center: trust.akka.io — 19 compliance standards; SOC 2 II + public

Logfire gives you tracing and evaluations — observe-and-evaluate on logs after execution. The EU AI Act expects enforcement inline to the runtime: immutable records witnessed as they happen, human override on running processes, pre-deployment classification, and a sealed audit artifact. Bolt-on observability cannot gate a deployment or classify a system before it ships. Akka embeds inline enforcement and covers pre-deployment governance.

Is PydanticAI cheaper because it is open source?

The framework is open source, but production means the Python infrastructure footprint plus a separate durable-execution engine and the memory, streaming, API, observability, and governance tiers you provision and operate. Akka's shared-compute model is up to 90% cheaper to operate for the same agentic transaction volume, on one fixed price.

SOC 3; annual pen tests, SBOMs, 40+ policies

Akka performance: akka.io/blog/go-slow-to-go-fast — Manulife up to 300% more concurrency, 30–50% faster; ~10T vs ~2T tokens/core; ~80% less compute than Python

Akka Specify (spec-driven delivery): akka.io / akka.io/llms.txt — you provide the specifications; Akka generates, tests, governs, delivers, and operates the system as a guaranteed outcome; one fixed price covering platform, infrastructure, tokens, training, delivery, and operations; delivered in weeks; continuous improvement via reinforcement learning and distillation (up to 80% lower token cost, higher accuracy)

Akka platform: 99.9999% availability, active-active HA/DR, sub-1 min RTO, zero byte RPO (contractual indemnities); 186 regulations / 877 controls / 288 with financial penalty; 100,000+ deployments / 18 years / 52 banks; profitable; Dell Technologies Capital